

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Теория информации

Определение избыточности сообщений.
Оптимальное неравномерное кодирование

Руководитель

подпись, дата

А.Н. Бочаров

инициалы, фамилия

Обучающийся БПИ24-02, 24151430

номер группы, зачетной книжки

подпись, дата

В.Т. Гордов

инициалы, фамилия

Красноярск 2026 г.

ЦЕЛЬ РАБОТЫ

Закрепление знаний по методам кодирования информации.

ЗАДАНИЯ

Влияние помех в канале связи описывается канальной матрицей, с помощью условных вероятностей $P(Y/X)$ и $P(X/Y)$, где X – источник информации, Y – приемник информации.

1. Используя текстовый файл из лабораторных работ №1-2, определить:
 - избыточность заданного текста, вызванную неравновероятностью появления символов в сообщении D_p ;
 - избыточность, вызванную статистической связью между соседними символами D_s ;
 - полную избыточность D .
2. Используя этот же текстовый файл, построить оптимальный неравномерный код, применяя:
 - метод Шеннона-Фано;
 - метод Хаффмана.
3. Определить для каждого метода среднюю длину символа исходного алфавита l_{cp} , коэффициент статистического сжатия K_{cc} , коэффициент относительной эффективности $K_{оэ}$.
4. Написать программу кодирования и декодирования исходного текста методом Шеннона-Фано и методом Хаффмана. В отчете привести отрывок текста объемом не менее 4-х абзацев, закодированного и декодированного обоими методами.

Вариант 5

каменного здания, мимо которого мы проходим.

Оборудование... слишком незнакомое, чтобы можно было его опознать. Несколь-

ко человек за столами. У одного в руках пробирка. Ха, химические опыты в виртуальном пространстве! Что-то новое. Имеет смысл, лишь если работа ведется

над очень ядовитыми веществами... или с бактериальными средами. Возьмем на заметку.

- Куда вы меня ведете? - интересуюсь у охранника. Лысина не оборачивается, но отвечает:

- К директору корпорации.

Имени он не называет, но и без того сказано многое. "Аль-Кабар" -

транснациональная корпорация, специализирующаяся на производстве лекарств, телефонной связи, кажется - еще на добыче нефти... Несмотря на весь арабский антураж, контролируется она из Швейцарии. Директор ее, Фридрих Урман - личность

слишком значительная, чтобы беседовать с каждым посетителем.

Теплая готовится встреча.

Мы останавливаемся у маленькой, увитой виноградом деревянной беседки, сзади меня

подталкивают, и я вхожу. Охранники остаются за порогом.

Внутри помещение куда больше, чем снаружи. Огромный павильон, в центре бассейн,

где медленно плавают сонные, блестящие рыбы. Рядом столик с двумя креслами.

Очень много цветов, я даже начинаю чувствовать запахи.

И никого.

Что ж... подождем. Сажусь в кресло.

Легкая мусть перед глазами... что и следовало ожидать. Сейчас прощупывают мой канал связи. Пытаются определить, откуда я пришел, объем информации, которую

могу принимать и передавать за секунду, присутствующие при мне программы...

Давайте, работайте. Шесть арендованных "на раз" роутеров, через которые пробегает сигнал. И все достаточно стойкие к взлому. А под конец - платный интернетовский гейт в Австрии, через который я и вошел в виртуальность.

Следы останутся, но никуда не приведут.

Можно в любой момент оборвать связь, "вышвырнуть" меня из квартала. Только что

им это даст... все вещички-программки, что находятся при мне, немедленно сработают. Мало что останется для изучения. А я им очень интересен, сомнений нет...

- Отслежен первый роутер, - сообщает "Виндоус-Хоум".

Быстро. Качаю головой - и в этот момент кресло напротив перестает быть пустым.

Господин Фридрих Урман пренебрегает арабским колоритом. Он в шортах, цветастой

рубашке. Пожилой, сухопарый, серьезный.

- Добрый день... дайвер, - произносит он. По-русски. Голос неестественный, пропущенный через программу-переводчик.

Вот и причина столь высокой чести.

- Боюсь, что вы ошибаетесь, господин директор.

- Когда полгода назад мы создали мост, это преследовало лишь одну цель, господин

дайвер. Обнаружить вас. Человек, находящийся в виртуальности, не смог бы его преодолеть, - Урман скупно улыбается. - Я первый раз вижу настоящего дайвера.

Один-ноль... не в мою пользу.

- А я первый раз вижу настоящего мультимиллионера.

- Вот видите, наша встреча уже принесла первые плоды.

"Виндоус-Хоум" шепчет:

- Отслежен второй роутер.

Урман хмурится - похоже, и ему что-то сообщают. Потом интересуется:

- Простите, через сколько компьютеров вы прошли на пути сюда?

- К сожалению, я не помню.

Урман пожимает плечами.

- Как я могу вас называть?

- Иван-Царевич.

Секундная пауза - потом улыбка. Ему объяснили.

- О, русский сказочный герой! А вы сами - русский?

- Разве это имеет значение?

- Да, вы совершенно правы... Господин дайвер, как я понимаю, вы проникли в наш квартал незаконно...

- Разве? - поражаюсь я. - Честно говоря, просто искал работу. Прочитал ваше объявление, прошел по мосту... подчинился этим странноватым охранникам. Один-один.

Фридрих Урман всплескивает руками.

- Да... действительно! Но у нас и нет претензий в ваш адрес, господин дайвер. Разве что странные вещи, которые вы носите с собой...

Медленно, демонстративно, вытаскиваю все из карманов. Расческа, носовой платок, маленькое зеркальце.

- Вот. Отдать вам меч?

Урман машет руками:

- Господи, к чему? Мы же не собираемся устраивать потасовок, верно? Давайте поговорим...

- Отслежен третий роутер.

- Как жаль, что времени на беседу остается все меньше и меньше, - вздыхаю я.

- Да, но времени всегда не хватает. Итак, господин дайвер, у меня есть основания полагать, что некоторые лица хотели бы получить ряд наших разработок. И даже ухитрились нанять дайвера... чтобы пожать чужие плоды.

- Яблочки, - уточняю я.

- Да, именно. У нас работает хороший русский программист, он сделал красивую форму для хранения информации... - Урман хлопает в ладоши, и рядом с нами воздух начинает мутнеть, сгущаться. Миг - и возникает маленькое деревце, усыпанное плодами. - Полагаю, что наибольший интерес вызывает вон то, маленькое зеленое яблочко на нижней ветке.

Я смотрю на вожденный плод. Он мелкий, незрелый и червивый.

- Как вы думаете, дайвер, сколько могли бы заплатить наши конкуренты за этот файл?

- Тысяч десять, - несколько завышаю цену.

Урман недоуменно смотрит на меня. Уточняет:

- Десять тысяч долларов?

- Да.

- Честно говоря, даже сто тысяч были бы невысокой наградой... Хорошо. Допустим,

я предлагаю человеку, пытавшемуся украсть файл, сто пятьдесят тысяч. При условии, что он начнет сотрудничать с нами... за нормальную, хорошую плату.

- Это что, лекарство от рака? - спрашиваю я.

Урман качает головой:

- Нет. Тогда бы оно не имело цены. Это лишь средство от простуды, но очень, очень действенное. Мы готовимся начать его производство, но лишь после того, как

будут распроданы запасы менее действенных лекарств. Так что вы скажете о моем

предложении?

ХОД РАБОТЫ

Для выполнения лабораторной работы было создано консольное приложение, написанное на языке Python. Исходный код программы представлен ниже.

Файл **header.py**

```
import math
import re
from collections import Counter

class TextAnalyzer:
    def __init__(self):
        self.alphabet_size = 32 # Длина используемого алфавита
        self.processed_text = "" # Текст после предобработки
        self.text_length = 0 # Длина текста после предобработки
        self.probs = {} # Словарь вероятностей символов  $p(x_i)$ 
        self.codes = {} # Словарь кодов для каждого символа
        self.hx = 0 # Энтропия источника  $H(X)$ 
        self.hyx = 0 # Условная энтропия  $H(Y|X)$ 

    def _clean_text(self, text):
        """Предобработка текста (32 символа, ь/ъ объединены)."""
        text = text.lower()
        text = re.sub(r'[^а-яё ]', '', text) # Убираем лишние символы
        text = text.replace('ь', 'ъ')
        return text

    def load_file(self, filename):
        """Загрузка текста из файла."""
        with open(filename, 'r', encoding='utf-8') as f:
            return f.read()

    def preprocess_text(self, text):
        """Предобработка текста и расчет вероятностей символов."""
        self.processed_text = self._clean_text(text)
        self.text_length = len(self.processed_text)
        counts = Counter(self.processed_text)
        self.probs = {char: count/self.text_length for char, count in counts.items()}
        return self.processed_text

    def analyze_redundancy(self, text):
        """Расчет избыточности текста."""
        # Энтропия источника  $H(X)$ 
        self.hx = -sum(p * math.log2(p) for p in self.probs.values() if p > 0)
        # Условная энтропия  $H(Y|X)$  через биграммы
        pairs = [text[i:i+2] for i in range(len(text) - 1)]
        pair_counts = Counter(pairs)
```

```

        h_xy = -sum(c/(self.text_length-1) * math.log2(c/(self.text_length-1)) for c in
pair_counts.values() if c > 0)
        self.hyx = h_xy - self.hx
        # Расчет избыточностей трёх видов
        h_max = math.log2(self.alphabet_size)
        dp = 1 - (self.hx / h_max)
        ds = 1 - (self.hyx / self.hx)
        d_total = ds + dp - (ds * dp)

        return dp, ds, d_total

```

```

def build_shannon_fano(self):
    """Построение кодов методом Шеннона-Фано."""
    # Сортировка по убыванию вероятности
    items = sorted(self.probs.items(), key=lambda x: x[1], reverse=True)
    chars = [x[0] for x in items]
    probs = [x[1] for x in items]
    self.codes = {c: "" for c in chars}

```

```

def divide(c_list, p_list):
    """Рекурсивное разделение алфавита на две группы
и присвоение бита на каждой итерации."""
    if len(c_list) <= 1: # базовый случай
        return

```

```

        min_diff = float('inf') # минимальная разница сумм
        best_split = 1 # индекс разделения
        # Поиск точки разделения
        for i in range(1, len(p_list)):
            diff = abs(sum(p_list[:i]) - sum(p_list[i:]))
            if diff <= min_diff:
                min_diff = diff
                best_split = i

```

```

        for i in range(len(c_list)):
            self.codes[c_list[i]] += "1" if i < best_split else "0"

```

```

        divide(c_list[:best_split], p_list[:best_split])
        divide(c_list[best_split:], p_list[best_split:])

```

```

divide(chars, probs)

```

```

def build_huffman(self):
    """Построение кодов методом Хаффмана."""
    from typing import Optional
    # Вспомогательный класс узла дерева
    class Node:
        def __init__(self, char, p):
            self.char, self.p = char, p # символ и его вероятность
            self.left: Optional['Node'] = None # левый потомок

```



```

        self.right: Optional['Node'] = None # правый потомок
# Инициализация узлов для каждого символа
nodes = [Node(c, p) for c, p in self.probs.items()]
# Построение дерева Хаффмана (пока не останется только корень)
while len(nodes) > 1:
    nodes.sort(key=lambda x: x.p)
    n1, n2 = nodes.pop(0), nodes.pop(0) # два узла с мин. вероятностями
    parent = Node(None, n1.p + n2.p) # новый узел-родитель
    parent.left, parent.right = n1, n2 # потомки нового узла
    nodes.append(parent)

# Генерация кодов
self.codes = {c: "" for c in self.probs}
def walk(node, code):
    """Рекурсивный обход дерева от корня к листьям."""
    if node.char: # базовый случай (есть символ = дошли до листа = сохраняем код)
        self.codes[node.char] = code
        return
    walk(node.left, code + "0")
    walk(node.right, code + "1")

if nodes:
    walk(nodes[0], "")

def get_efficiency(self):
    """Параметры оптимальности текста."""
# Средняя длина символа кодового алфавита
l_cp = sum(self.probs[c] * len(code) for c, code in self.codes.items())
# Коэффициент статического сжатия
h_max = math.log2(self.alphabet_size)
k_cc = h_max / l_cp
# Коэффициент относительной эффективности
k_oe = self.hx / l_cp
return l_cp, k_cc, k_oe

def encode(self, text):
    """Кодирование текста по построенным кодам."""
    clean = self._clean_text(text)
    return "".join(self.codes.get(c, "") for c in clean)

def decode(self, binary_str):
    """Декодирование бинарной строки в текст."""
    inv = {v: k for k, v in self.codes.items()}
    res, temp = "", ""
    for bit in binary_str:
        temp += bit
        if temp in inv:
            res += inv[temp]
            temp = ""
    return res

```

Файл **main.py**

```
from header import TextAnalyzer

def demonstrate_text_processing(analyzer):
    if not analyzer.processed_text:
        print("Текст не загружен!")
        return

    # Извлекаем минимум 4 абзаца
    paragraphs = [p.strip() for p in analyzer.processed_text.split('\n') if p.strip()]
    if len(paragraphs) < 4:
        sample_text = "\n\n".join(paragraphs)
    else:
        sample_text = "\n\n".join(paragraphs[:4])
    print(f"Извлечено абзацаов из текста: {len(paragraphs)}.")

    print("\n" + "="*40)
    print("КОДИРОВАНИЕ И ДЕКОДИРОВАНИЕ ТЕКСТА")
    print("="*40)
    print(f"ИСХОДНЫЙ ТЕКСТ:\n{sample_text}\n")

    methods = [
        ("Шеннон-Фано", analyzer.build_shannon_fano),
        ("Хаффман", analyzer.build_huffman)
    ]

    for name, build_func in methods:
        build_func() # Строим коды
        encoded = analyzer.encode(sample_text)
        decoded = analyzer.decode(encoded)
        l_cp, k_cc, k_oe = analyzer.get_efficiency()

        print(f"--- МЕТОД: {name} ---")
        print(f"Средняя длина символа кодового алфавита (Lcp): {l_cp:.4f} бит/симв")
        print(f"Коэффициент статического сжатия (Kcc): {k_cc:.4f}")
        print(f"Коэффициент относительной эффективности (Koe): {k_oe:.4f}")
        print(f"Закодированная последовательность (первые 100 бит):\n{encoded[:100]}...")
        print(f"Результат декодирования:\n{decoded}")
        print(f"Проверка целостности: {'УСПЕШНО' if decoded == sample_text else 'ОШИБКА'}")
        print("-" * 30)

def main():
    analyzer = TextAnalyzer()

    while True:
        print("\n--- ЛАБОРАТОРНАЯ РАБОТА № 3 ---")
        print("1. Загрузить текст из файла")
        print("2. Задание 1: Расчет избыточности (D)")
        print("3. Задания 2-4: Кодирование/декодирование текста")
```

```

print("0. Выход")

choice = input("Выбор: ")

if choice == "1":
    fname = input("Введите имя файла (по умолчанию 'input.txt'): ") or "input.txt"
    analyzer.preprocess_text(analyzer.load_file(fname))
    print(f"Текст загружен ({analyzer.text_length} символов)")

elif choice == "2":
    if not analyzer.processed_text:
        print("Текст не загружен!")
        continue

    dp, ds, d = analyzer.analyze_redundancy(analyzer.processed_text)
    print(f"Избыточность, вызванная неравномерностью распределения символов (Dp): {dp:.4f}")
    print(f"Избыточность, вызванная статистической связью между соседними символами (Ds): {ds:.4f}")
    print(f"Полная избыточность (D): {d:.4f}")

elif choice == "3":
    demonstrate_text_processing(analyzer)

elif choice == "0":
    break
else:
    print("Неверный ввод.")

if __name__ == "__main__":
    main()

```

В коде представленной программы был реализован класс **TextAnalyzer**, предназначенный для работы с текстом.

Взаимодействие с функциональными методами происходит с помощью консольного меню (Рисунок 1).

```

--- ЛАБОРАТОРНАЯ РАБОТА № 3 ---
1. Загрузить текст из файла
2. Задание 1: Расчет избыточности (D)
3. Задания 2-4: Кодирование/декодирование текста
0. Выход
Выбор: 

```

Рисунок 1 — Меню программы

При выборе 1 пункта программа загружает текст из выбранного пользователем файла и выполняет его предобработку (Рисунок 2): убирает лишние символы (приводит к 32-буквенному алфавиту), считывает длину текста, формирует таблицу вероятностей каждого символа (словарь вида [символ: вероятность]).

```
Выбор: 1
Введите имя файла (по умолчанию 'input.txt'):
Текст загружен (5032 символов)
```

Рисунок 2 — Загрузка файла в анализатор

Задание 1

При выборе 2 пункта программа последовательно высчитывает:

- Энтропию источника $H(X) = - \sum_{i=1}^n p_{xi} * \log_2 p_{xi}$;
- Энтропию объединения $H(X, Y) = - \sum \sum p_{ij} * \log_2 p_{ij}$;
- Условную энтропию $H(Y/X) = H(X, Y) - H(X)$;

Далее происходит расчёт показателей избыточности текста (Рисунок 3):

- Избыточность, вызванная неравномерностью распределения символов $D_p = 1 - (\frac{H(X)}{H_{\max}})$;
- Избыточность, вызванная статистической связью между соседними символами $D_s = 1 - (\frac{H(Y/X)}{H(X)})$;
- Полная избыточность $D = D_s + D_p - D_s D_p$.

```
Выбор: 2
Избыточность, вызванная неравномерностью распределения символов (Dp): 0.1094
Избыточность, вызванная статистической связью между соседними символами (Ds): 0.1992
Полная избыточность (D): 0.2868
```

Рисунок 3 — Расчёт показателей избыточности текста

Задания 2-4:

При выборе 3 пункта программа предварительно извлекает из текста первые 4 абзаца (Рисунок 4).

```
Выбор: 3
Извлечено абзацов из текста: 99.

=====
КОДИРОВАНИЕ И ДЕКОДИРОВАНИЕ ТЕКСТА
=====
ИСХОДНЫЙ ТЕКСТ:
каменного здания мимо которого мы проходим

оборудование слишком незнакомое чтобы можно было его опознать несколько

человек за столами у одного в руках пробирка ха химические опыты в

виртуальном пространстве чтото новое имеет смысл лишь если работа ведется
```

Рисунок 4 — Извлечение и предобработка фрагмента текста

Далее производится последовательное выполнение алгоритма кодирования и декодирования для обоих методов (Шеннона-Фано и Хаффмана), сравнение декодированного текста с исходным, а также вычисление следующих метрик:

- Средняя длина символа кодового алфавита $L_{cp} = \sum_{i=1}^n p_i l_i$;
- Коэффициент статического сжатия $K_{cc} = \frac{H_{max}}{l_{cp}}$;
- Коэффициент относительной эффективности $K_{oz} = \frac{H}{l_{cp}}$;

Результаты расчётов представлены на рисунках 5 и 6.

```
--- МЕТОД: Шеннон-Фано ---
Средняя длина символа кодового алфавита (Lcp): 4.4922 бит/симв
Коэффициент статического сжатия (Kcc): 1.1130
Коэффициент относительной эффективности (Koe): 0.9913
Закодированная последовательность (первые 100 бит):
0011110100101010111000100011000001111011100010001000101010000111001000111010100111010101101110011111...
Результат декодирования:
каменного здания мимо которого мы проходим

оборудование слишком незнакомое чтобы можно было его опознать несколько

человек за столами у одного в руках пробирка ха химические опыты в

виртуальном пространстве чтото новое имеет смысл лишь если работа ведется
Проверка целостности: УСПЕШНО
```

Рисунок 5 — Применение алгоритма Шеннона-Фано

```

--- МЕТОД: Хаффман ---
Средняя длина символа кодового алфавита (Lcp): 4.4855 бит/симв
Коэффициент статического сжатия (Kcc): 1.1147
Коэффициент относительной эффективности (Koe): 0.9928
Закодированная последовательность (первые 100 бит):
0100010011000110110110011011110010001111110001001010011001011000111010011101000100111000111111100100..
Результат декодирования:
каменного здания мимо которого мы проходим

оборудование слишком незнакомое чтобы можно было его опознать несколько

человек за столами у одного в руках пробирка ха химические опыты в

виртуальном пространстве чтото новое имеет смысл лишь если работа ведется
Проверка целостности: УСПЕШНО

```

Рисунок 6 — Применение алгоритма Хаффмана

Так, по результатам вычислений были получены следующие данные:

- Избыточность, вызванная неравномерностью распределения символов (D_p): 0.1094;
- Избыточность, вызванная статистической связью между соседними символами (D_s): 0.1992;
- Полная избыточность (D): 0.2868;

Метод **Шеннона-Фано**:

- Средняя длина символа кодового алфавита (L_{cp}): 4.4922 бит/симв;
- Коэффициент статического сжатия (K_{cc}): 1.1130;
- Коэффициент относительной эффективности (K_{oe}): 0.9913;

Метод **Хаффмана**:

- Средняя длина символа кодового алфавита (L_{cp}): 4.4855 бит/симв;
- Коэффициент статического сжатия (K_{cc}): 1.1147;
- Коэффициент относительной эффективности (K_{oe}): 0.9928.

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что лежит в основе определения избыточности?

В основе лежит разность между максимально возможной энтропией алфавита H_{max} реальной энтропией источника H . Избыточность показывает относительное количество «лишних» символов, которые не несут новой информации, но присутствуют в сообщении.

2. Какие частные избыточности вы знаете?

Избыточность из-за неравновероятности D_p (одни символы встречаются чаще других) и избыточность из-за статических связей D_s (зависимость между определёнными символами).

3. В чем заключается оптимальное неравномерное кодирование?

Это способ кодирования, при котором частым комбинациям присваивают короткие коды, а редким — длинные. Цель — минимизировать среднюю длину кодового слова l_{cp} , максимально приблизив к значению энтропии источника.

4. Как проводится кодирование информации методом Шеннона-Фано?

1. Символы алфавита выписываются в порядке убывания их вероятностей.
2. Алфавит делится на две группы так, чтобы суммы вероятностей в этих группах были максимально близки.
3. Первой группе присваивается символ «1», второй — «0».
4. Процесс рекурсивно повторяется для каждой группы, пока в них не останется по одному символу.

5. Как кодируется информация методом Хаффмана?

1. Символы алфавита выписываются в порядке убывания их вероятностей.
2. Два символа с наименьшими вероятностями объединяются в один узел с суммарной вероятностью.
3. Новый список снова сортируется, и процесс повторяется до тех пор, пока не останется один корень.
4. Коды строятся при движении от корня к листьям (путь влево — 0, вправо — 1).

6. Как определяется средняя длина кода?

l_{cp} — это мат. ожидание длины кодовой комбинации: $l_{cp} = \sum_{i=1}^n p_i l_i$; p_i — вероятность символа, l_i — его длина кода в битах.

7. Как вычисляются коэффициенты статистического сжатия и коэффициент относительной эффективности?

Коэффициент статического сжатия $K_{cc} = \frac{H_{max}}{l_{cp}}$;

Коэффициент относительной эффективности $K_{оэ} = H/l_{cp}$;

8. Как формируется информационная модель канала связи?

Модель представляется как совокупность источника сообщений X , кодера, канала передачи (описываемого матрицей переходных вероятностей $P(Y/X)$), декодера и приемника Y . Основная характеристика здесь — условная энтропия $H(Y/X)$, описывающая помехи в канале.

9. Как определяется пропускная способность канала связи?

Пропускная способность C — это максимально возможное количество информации, которое канал способен передать в единицу времени: $C = \max_{P(X)} [H(X) - H(X/Y)]$. Определяется как максимум взаимной информации между входом и выходом канала по всем возможным распределениям вероятностей на входе.

10. В чем заключаются недостатки оптимального кодирования?

1. **Чувствительность к ошибкам:** при потере или искажении одного бита декодер может потерять синхронизацию, и весь последующий текст будет расшифрован неверно.
2. **Сложность реализации:** требуется наличие таблицы кодов (словаря) на обеих сторонах связи.
3. **Переменная длина:** затрудняет разделение сообщений на блоки в реальном времени.

11. В чем суть основной теоремы Шеннона о кодировании для дискретного канала без помех?

Если энтропия источника H меньше пропускной способности канала, то существует такой метод кодирования, который позволяет передавать сообщения со скоростью, сколь угодно близкой к C , с вероятностью ошибки, стремящейся к нулю. Это достигается путем полного устранения избыточности.

12. В чем суть теоремы Шеннона для побуквенного кодирования?

При кодировании символов источника по отдельности средняя длина кодового слова l_{cp} всегда ограничена снизу энтропией источника, а сверху — величиной энтропии плюс один бит (для двоичного кода):

$$H(X) \leq l_{cp} < H(X) + 1$$

ВЫВОДЫ

В ходе выполнения лабораторной работы были закреплены теоретические знания по методам кодирования информации. Были получены практические навыки по определению показателей избыточности текста, а также кодированию и декодированию текста.

Исходя из полученных результатов вычисления, можно сделать следующие выводы:

Задание 1: Полная избыточность текста D составила 28.68%, что складывается как из неодинаковой частоты появления букв, так и из наличия устойчивых буквосочетаний в естественном русском языке.

Задания 2-4: Использование методов кодирования Шеннона-Фано и Хаффмана позволило сократить среднюю длину символа с 5 бит/симв (максимальная энтропия) до 4.49 бит/симв в среднем.

Помимо этого, метод Хаффмана показал большую эффективность по сравнению с методом Шеннона-Фано: 99.28% против 99.13% соответственно, что также выражается в меньшей средней длине символа.

Проверка кодирования и декодирования текста подтвердила свойство префиксности полученных кодов — текст восстанавливается однозначно и без потерь.